

ActInf MathStream 005.1 ~ Cristian Bodnar "Topological Deep Learning: G

MathStream | Cristian Bodnar | 1:05:46

Hello and welcome. It's July 17th, 2023. We're here in Active Inference Math Stream number 5.1 with Chris Bodnar on topological deep learning, graphs, complexes, and sheaves. So thank you for joining, Chris. Looking forward to your presentation and discussion. Yeah, thanks. Thanks all for having me. So yeah, as I was just saying, this is my PhD thesis, which I finished a couple of months ago. It's also kind of a public online. So, you know, if you want to go into the details, just kind of look this up on the internet and you should be able to find it easily. Obviously, there's lots of stuff in there. So I kind of try to give an overview today and maybe also go in a little bit more detail in certain aspects, since there's not a lot of time to go through everything. So now I'm a Microsoft researcher. So this is some basic old work that I did in the past and when I was at University of Cambridge. All right. So let's get started. Right. So let's start very easily. Now I'm actually not sure exactly what's kind of the background of the people who are watching, but in machine learning, there's all these kind of subfield that emerged a few years ago, which is called geometric deep learning, which is essentially looking at how to apply these kind of deep learning neural network architectures on data, living on all sorts of, you know, kind of structures or geometries or spaces, if you want. And this has a lot of applications, especially in the life sciences. And there's kind of been a lot of instances of this in kind of, you know, very famous publications, I don't know, which you see here, but, you know, just to give some examples, for instance, if you have proteins or molecules or things like that, they usually represent it as graphs. And you kind of have some data living on these graphs, like kind of the properties of certain atoms and so on. So, so these kind of things. And so far, these kind of spaces or these kind of problems, learning problems, if you want, they have been approached mostly kind of with a geometrical mindset, as the kind of name of the of the subfield also mentions. But something that, you know, I would argue is that geometry is not everything that you need. And there's kind of other non geometrical aspects when you are in such a setting. And this is kind of quite obvious once you realize that the spaces that kind of show up in in the field and in many applications, they are very heterogeneous. So as I mentioned, for instance, you could have graphs that, you know, could represent anything in this case, it's the caffeine molecule that you see here on the left. And you want to, you know, have some models that predict certain properties of this molecule and so on. But for instance, you can have grids, and we see grids all the time, and data living on grids and are referring to images, videos, they are all kind of, you know, pixels living on a grid. And then you can have more sophisticated things, you could have some meshes, for instance, they're all over in computer graphics, and then you could have some sort of manifold. So for instance, if you're doing maybe weather modeling or something, you're, you know, we live on a sphere, topologically speaking. So you might want to model your data as living on a sphere, and so on. But nonetheless, even if these spaces are kind of geometrically kind

of heterogeneous, and some of them don't even have a geometrical structure in kind of a strict mathematical sense, they all have what's called a topological structure, which is kind of like, kind of a weaker kind of structure. But it's kind of more general. And I'm going to talk a bit, in a few seconds about what that means. But in general, when you do kind of mathematical physics, you kind of have a ladder of structures where kind of, you know, you kind of keep building on top. And the more structure you have, the more sophisticated things you can do, and so on. And kind of at the basis of this diagram, just the sets, just kind of a collection of elements with no kind of extra structure. And then you kind of keep going up in this ladder, and you add stuff on top of sets, and so on. And as I was saying, kind of most of the work is kind of focused on maybe the top levels of this hierarchy. But it's kind of topological level, which are kind of, you know, the weakest kind of level, you can add on top of sets has kind of been neglected to a large extent. And part of what I've been doing in my PhD thesis was essentially looking at this kind of learning problems on these kind of spaces from a more topological perspective, and, you know, kind of try to fill in these blanks.

So this is kind of the overview. And now, okay, so if we are to adopt this topological perspective, well, what would that actually mean? Or how would that look like? I guess it could look in different ways. But in what I've done in my thesis, it looks kind of like this. So we have horizontally, we kind of have a space that could be anything, there's just kind of an abstract space, you know, it could be a grid or things like we've seen in the previous example, and kind of vertically attached to the regions of the space, we have data. So data is kind of this vertical component, and you kind of see these flags kind of being, you know, kind of anchored in these regions. So that kind of signifies, you have some data that's kind of associated with that region. So that's kind of the high level perspective. And I'm going to make that a bit more concrete, a bit later. And there's kind of two essential things about this picture. The first thing is locality. So the data is attached to some regions of this space, this topological space. And in that sense, it's local. So it's kind of associated with the region. And maybe to give a concrete example, if you kind of have a temperature sensor somewhere in space, right, you could think of whatever that sensor is measuring is kind of a property of kind of the immediate surrounding of, you know, around that sensor. So it's kind of describing some property of a region in space. So that'd be kind of a concrete example. And another kind of axiom that that we're starting with is that the space has structure. So the space has kind of, it's kind of made up of various regions, and these regions intersect in various ways. So the, and that implicitly also makes our data structure because the data is attached to these regions. So, so there is kind of some, some structuring in the data. All right. So that's kind of the picture. And actually, many of these things relate to category theory. I'm not going to go in depth into this because it's kind of sophisticated and I'm not an expert myself in category theory, but kind of the high level here is that category theory is kind of a nice way to translate between different structure in mathematics and kind of, you know, discuss about properties of certain kinds of objects and translate that to some different kinds of objects and find all these kinds of relations and connections. And a concrete example is, for instance, if you want to study some manifolds, some surfaces, you could associate some groups to

these surfaces and then any sorts of relations between these kind of surfaces also translating some relations about these groups or some other algebraic structures. So you could study these manifolds by doing algebra instead of doing, you know, geometry or topology or something else. So also in this case, this kind of manifests in the fact that we kind of translate these from spaces to data. So kind of, because we associate the regions in a space, certain kinds of data, this is kind of how this translation manifests in what I've just described in the previous picture. So you could think of this as some sort of translation or mapping from, from spaces and regions in that space to kind of data attached to that space. But yeah, I'm not going to go in a lot of detail into this, but it's just kind of to, you know, to keep in the back of your mind that there's, there's this stuff lurking in the background. All right. So this is, I promise this is the only math definition I'm giving in this talk and then, and then I'll stop. But just because I'm mentioning topological space, spaces quite often, I just wanted to kind of give this axiomatic definition, which might sound sophisticated, but I have a picture at the end and hopefully it'll be clear. So yeah, so what is it? So it's just a set, as I was saying, we start with sets and we put stuff on top, right? So you start with a set, and then you also have a collection of subsets of the set, called the open sets that need to satisfy certain axioms. So you could think of these open sets as kind of regions of the space, very informally speaking. So something that kind of has to be satisfied is that, well, the empty set and the set itself need to be open sets. So in some sense, you could think of this saying the set itself is a region of that space, very informally, which, you know, it's kind of, let's say, obvious. And then there's some kind of constraints about intersecting and taking unions of this region. So if we take the intersection of two regions, we should get another one of these regions. And if we take a union of these regions, we should take we should get another region. And there is some constraints again, like, okay, how big this intersection should be, there should be finite intersections, but you could have infinite unions, but that's a technicality. And we can just keep anyways, but to see a picture. So on the left, you just see the set, X itself. And here I put like a potential neighborhood structure, kind of like open set structure on this space. So we have an open set here, another open set V . By this axiom, the intersections would also be an open set. So you see this intersection in the middle being another open set. And then the set itself is another open set. So it's just kind of a splitting stuff into regions kind of, you could think of it like that. All right, so this is a topological space. And now let's add data, right? So we mentioned that we have data, and then we put, sorry, we have a space and then we add data on top. So, so far, we've seen how a topological space looks like. Now, let's add this kind of vertical stuff, these flags that you saw before, we just put some data on top of these regions. And if we put data on all the regions of the space, on all the open sets, we get these structures that in category theory, like in algebraic topology, also geography, they are called pre-sheaves, which sounds very fancy, but all is just kind of a definition of what I was already describing. Essentially, you have some data for each region, these, these F of, so for instance, for region U , you have these F of U , which is kind of the data attached to region U . So you could think of

F of U, some, some set with, with describing the data that lives there in that region. But there's also an extra thing, you have some, some sort of maps going between these kind of pieces of data. And these are called restriction maps. And why is that? It's kind of, they provide you a way to kind of zoom in if you want, right? Like you have, you have the data attached to the whole set X, and then you could think, okay, how do I take this data? How do I go from this data to data on a smaller region on X? There's kind of a way to zoom in on that data in essentially, and I'm going to show some, some example in a second. So these are called pre-sheaves. And just to see an example, our space here is kind of one of the simplest kind of space you could think of, it's just 1D or horizontal line, right? It's just the real line. And then you have some regions which are just given by open intervals on, on this, you know, on the real line. And then some pieces of data could be functions, like continuous functions on, on those regions. So here's like some sample data on, on this first interval. Here's another function on the second interval. Here's some data on this third interval. And actually in this case, it happens that all these functions agree on the overlap. So where these regions overlap, they take the same values and we can actually glue them together on a single, in a single function over the entire region, right? So, so this is just an example of, of a pre-sheaf and it's called the pre-sheaf of continuous functions. So our data in this case is continuous functions and the space is just the real line. And we put these functions on top of the real line. But it turns out because of this kind of special property that we can kind of glue data and we uniquely get some other piece of data, right? We, we can take these three pieces. It's just exactly like a, like a puzzle, right? We put these things, things together and we get a fourth thing, which is kind of a single function where we just overlap these, these functions, right? And these pre-sheaves that satisfy this kind of properties where you can kind of glue them to get a unique piece of data. They're called sheaves. And basically the pre-sheaf of continuous functions is actually a sheaf. So this is kind of a way to formalize data attached to these things. It's going to get less technical in a second. So just to give more examples. So for instance, in this way, we could describe data on a sphere. And let's say if this data is just some vector field on a sphere. So let's say if this is earth, right, this could be some wind vector field, right? Like if we do weather modeling or something, you just have a vector field describing the wind on the surface of earth, right? And you might want to do some machine learning on top of this, where this kind of vector field has a sheaf structure. And you could think of it as a sheaf because, you know, if you have like some vector field on the red region, a vector field on the yellow region, I can kind of glue them together uniquely if they overlap, if they agree on the overlap, and we get the vector field on this bigger region. But something that's quite nice is that even if we have a very, very different kind of space, namely a graph, which is very different from a sphere in all points of view, we can still apply the exact same kind of axioms and terminology and kind of definitions. And we can have a sheaf over a graph. So in this way, we could have, for instance, some features associated to the nodes of the graph, some features associated with the edges of the graph. And there's another node which has its own features. And this is actually the exact setting we have in graph machine learning. So this is quite nice what this kind of topological

perspective allows us to do is, you know, we kind of have a unified way of thinking if you want, about very kind of heterogeneous spaces, and we can model on all of them, data attached to them by using this kind of sheet terminology, and other ways as well, but I'm not going into that in this talk.

All right, so this is kind of an overview of what I've been doing in my thesis. And just to kind of dive a bit deeper into this, I just wanted to go into one paper that we did at NeurIPS last year. Yeah, so this was last year on what's called shift diffusion. So essentially, how can we use what I've just described to do some useful stuff when doing machine learning on graphs. All right, and this was a collaboration with Francesco Di Giovanni, Ben Tamberlain, Pietro Leo, my advisor, and Michael Bronstein.

Okay, so before I dive into this, I just want to give some background in case people are not familiar with this. So the kind of favorite architecture of people doing machine learning on graphs, these days are these things called graph neural networks, which are actually very simple kind of models. So in this setting, you have some features. So each node in your graph will have some features. This is what is h vector here, the node. So it's the vector h associated with node a at layer or time t , whatever. So you have some features for each of these nodes. And what you're doing, each of, if you're at a certain node, you want to kind of compute a new representation or a new features for this node at the next layer. So essentially, you're learning representations. And what the graph neural networks are doing, this node will receive a message from all the other

nodes that are neighbors with this node. And this message can also be passed, you know, can use some neural networks in there. But essentially, some processing of these features of the neighbors. And these are aggregated into this message. So here in green, and then there's this is passed through some update function that combines the message from the neighbors with the old representation of this node. And it gives you a new representation at the next layer. And this happens for all the nodes, right. So then you get some new representations for this node. And this is one layer. And then you kind of keep repeating this for as many layers as you like. So this is kind of how you do deep learning on graphs. It's kind of a very, very simple recipe. And most models actually vary in the way they kind of compute these messages and in the way this update function is designed. But that's kind of the parameters they most of these models use. Otherwise, they all kind of respect this framework and work in this kind of particular way. And to give you maybe an example for why you would want to do this, you might want to do node classification. This is kind of a classic problem in graph machine learning. There are others, but I'm just going to talk about this because it's easier. So you have a graph. And this graph has nodes that have different labels here. There's just two kinds of labels, this orange and blue. And you have some edges between these nodes. And what you want is you want to do this kind of message passing that I was describing to compute some representations for these nodes where you can easily classify the blue and orange nodes. Now, something that's quite interesting is that for many kind of graph neural networks, depending on the properties of these graphs and how these kind of different nodes are connected, their performance might vary quite a lot. So in particular, they're affected by this property called heterophily. So this is kind of a measure of how much opposites attract if you want. So it has a very simple formula. Basically, you take the number of edges

between orange and blue nodes and you divide by the total number of edges. So basically, you kind of check how many connections we have in this graph between things that are quite opposite to each other versus connections that are between similar kind of nodes. So if you have a lot of these kind of heterogeneous connections, then you have very high heterophily. And it turns out that many graph networks actually struggle in that setting, it's very hard to classify things in that setting. And intuitively, you could kind of also, you know, figure out why, because, you know, you could easily apply to some kind of reasoning, where, you know, oh, this node looks a lot like these other nodes connected to so they kind of must be in, you know, in the same community, if you want, or in the same label. But it's much harder to do that when, you know, all things are kind of different from each other. And these communities kind of don't form, right? Even visually, if you see a graph, and it has some nicely clustered communities, you know, it's quite easy to draw a line between those and say, oh, this is a community is another community. But if things are very mixed, then it's quite challenging. And it turns out it's also challenging for these models, not just for kind of our intuition, right? When, you know, we would have to do this.

So this is kind of some problem where this perspective, this topological perspective I was mentioning, will be used to, you know, do some useful stuff. Okay, so coming back to the sheaves on graphs. And at this point, I think you can largely forget what I mentioned in the introduction. Or if there's something you misunderstood there, you know, we kind of start from zero a bit here. So there's no problem. So on the left, you just have a graph, which is kind of the incident structure of a graph, I just wrote here the simplest possible graph that has two nodes V and U . And then there's an edge between them. So this is just a graph with one edge. That's all that's going on here. And I've just represented it by kind of in this kind of incidence structure kind of way, right? Node V is incident to node E and node U , sorry, edge E , and node U is incident to edge E . So this is just an incident structure. And what is kind of triangle symbol showing is just this incident structure is just a way to symbolize this incidence relation if you want. Okay, so this is just a graph, right? And a way we can kind of think of sheaves on graphs is just mapping these graph structures. So this is kind of this categorical theory translation, we translate this graph into something else, which looks, you know, very similar, the structure is kind of the same, it's just kind of the meaning of these things change. So for each node V , we have here, this will be a vector space. So F of V is a vector space. For each node U , we have F of U , which is another vector space. For each edge E , we have this F of E , which is another vector space. So all nodes have their own vector spaces. And the features associated to those nodes leaving those vector spaces. So basically, for each node, we have a vector space of features, that's all that's going on so far. And also these arrows that these incidence relations also translate into something. And they translate into the obvious thing linear maps. So if these are vector spaces, then these things should be linear maps or just some matrices essentially, right? So for each arrow, you see here, we have a matrix. And yeah, and something that I'll argue and show in a few slides is that

basically message passing is very similar on graphs is very similar with group actions in group theory.

So let me explain exactly why. So we kind of can think of what we have on the left, these arrows in from the incidence relation, we could think of these arrows as kind of some buttons we can press. So what do I mean by that?

So if we have this node V on the left, right, and this E , now if we have some features, some feature living in F of V , right, we could just kind of press this arrow button here. And then if we multiply this matrix by this feature, we will get an edge feature. So it's kind of like, if you go along this arrow, this matrix will multiply this feature and this vertex feature, and it will give you an edge feature.

So you can think of these arrows as kind of giving you some sort of actions that you can play with to move features from vertex to edge and edge to vertex. So in this case, it's kind of a left action, right?

So this is what I'm saying, I'm taking this arrow, which is this one here, and I act on some features of node V . So this H of V that lives here, right? And how I do that is I just take this matrix, this matrix associated with this arrow, and I multiply this vector H of V . So just matrix times vector, that's all, and then we get an edge feature. So this is just kind of a way to move from here to here.

So this already kind of looks a bit like message passing, right? We're kind of passing a message from this vertex to this edge. But now we also need to pass a message from this edge to these other vertex U . So we kind of, you know, we need to get from V to U . And we did that by passing through E . So by doing that, we could do that by kind of going in reverse. So we could have a right action, where instead of applying this matrix, we apply its adjoint matrix. So that's just the transpose matrix.

So instead, if we want to go from here to here, instead of applying this matrix, we apply its transpose because we want to go the other way around. So this is, this is a, if we compose these things, then we can move features from V to U . So this is just a way that kind of, we can apply these actions to do message passing. And these are called shift actions or pre-shift actions. And I'm going to now show what's kind of the relation with, between this and what we have in group theory. So one way to represent the group is by kind of having some sort of graph, like here on the left. So we kind of have some star object, it is just kind of a dummy thing there. But all the group structure is in these arrows, right? So for each group element, so let's say this G is a 90 degree rotation, for instance, that's, let's say we do have a group of rotations, just to have some concrete example, these, so this arrow could correspond to a 90 degree rotation, we have another arrow that does the opposite minus kind of 90 degree rotation, that's the inverse of that transformation. So this is kind of the structure of this group. And if we have, we also do this kind of similar kind of translation, as we've just seen.

So basically, basically, we define a pre-shift on this group, we map this star to a vector space. So the star kind of replaces the vertex we had before, now we just have a single vertex. And it's just these arrows we have. So the star is not this vector space that you showed here in blue, right? And now if we actually do group actions, which are kind of a very well established concept in group theory. So essentially, well, for instance, if you want to act on this vertex, sorry, not vertex on this vector V , right here, you have a vector in this vector space, and you want to act on it by this group transformation G . So essentially, you want to press this arrow, so you apply some action on it.

Then what you do is, well, because of this translation, this G has been mapped to some matrix,

which is the rotation matrix, the corresponding rotation matrix. And you apply this rotation matrix on V , and you get like a 90 degree rotation here. So this is what's going on, this kind of vertical vector is showing the rotated vector. So this is completely analogous with what we've seen on the previous slide. This is how kind of sheaves connect these kind of actions. So essentially, what you could think of as message passing is same as group actions in group theory, but you just replace this group with a graph. So it's kind of analogous to that, right? So it's just kind of a different kind of translations, where we replace the object on the left. Now it looks like this is a graph, this is a group, right? So but kind of the rest stays exactly the same. So you could, and this kind of gives us a way to formalize in a way by looking in this topological perspective, to connect all these kind of symmetries and things that have been explored quite a lot in machine learning, to message passing on graphs, and to see one way in which in which they are related. Okay, so now you might say, okay, this was all very sophisticated and nice. But you know, what is this going anywhere, basically? And I'm just going to show you kind of a very short example there, there's more, but you know, the time is limited. And something we showed is that, as I was saying in the beginning, many graph neural networks kind of struggle in these heterophilic graphs. And what we've shown is that no matter how heterophilic or kind of weird your graph is, you can always kind of find some shift structure, essentially kind of a message passing neural network, that if you use sufficient layers, it will be able to disentangle the classes of the of the nodes, right. So just to show you in this picture, what you have here on the very far left, the colors of the nodes, they show the class. So we have three, there's three colors here, so three classes, right. And this is kind of the graph in the beginning, and the position of the nodes in this box denotes the features. So that's a way to kind of just to visualize the features. And of course, the position, the 2d position is actually the 2d feature vector of each node. And you can see in the beginning, everything is kind of super messy and entangled, right? Like, if you want to classify these nodes, it's kind of very hard because their representations, their initial representations are very messy and kind of intertwined. But as we stack more layers of a particular kind of sheath or a message passing model, you see how progressively these classes get, you know, kind of more disentangled and more disentangled at each new layer. And so kind of these representations kind of collapsed, and they form this kind of clustering, right. And then when you get with something like at the end, you can kind of see these three communities very clearly, and it's extremely easy to separate. And kind of the essence behind these results was, we showed for different kinds of problems, what sorts of sheath or message passing models you need by using the theory to solve kind of problems. And this is quite important because it kind of shows you some important bits and pieces in the architecture that you might want to kind of change or use in order to solve certain kinds of problems. And we also had some sort of impossibility results. So if you use a graph mu network of some kind, you can't solve this problem or you'll struggle to solve this problem. And we also saw, okay, if you use some more general ones, then you might have a chance. So this is kind of some, the, you know, high level view behind this theoretical stuff. And what we actually doing in practice is to essentially learn these message passing functions or to learn the sheath or these matrices.

So in practice, like when someone gives you a null classification task, it's very hard to know beforehand what exactly is the right sheath or the right message passing model to solve that task. And what we do is actually we learn that from data. So we learn these matrices that do the message passing. We learn them from data by passing some, using some neural networks, which are shown here in red. And then you learn how to kind of transfer features between these vector spaces and kind of move them around. So this is just showing how these vectors, which are features of these nodes and edges, how they're kind of moved around by kind of going through these matrices, just some matrix multiplications. So that's kind of the high level view behind this model. And we evaluated this on some kind of real world heterophilic data sets where you have to classify nodes based on kind of various communities or different kinds of labels. And these data sets going from right to left, they are getting more heterophilic. So in some sense, more challenging for classic architectures. And our models, which are kind of inspired by all this stuff that I mentioned, they score quite highly in these benchmarks. And at the same time, they also revealed some or justified some various choices that other models in this space have done, but maybe they were not so well justified, or maybe they had different kind of motivations. We also managed to kind of show why various things they were already doing, why they made sense from the point of view of this kind of theory. All right, well, that that's all, all I had. Yeah, thanks for listening. And yeah, happy to chat more about this. And also have lots of backup slides in case, you know, depending on how far we can venture off with these questions.

David Pérez, Ph.D.: Cool. Well, awesome work. Thank you for the presentation. For people who are in the live chat, they can write some questions, but there's many things I think we could talk about. So I want to start with reading a quote from an abstract of the paper by Van der Laar, Kudel, and De Vries, just to kind of ground this in the active inference context and really justify why the message passing approaches that you are

describing are helping in the active inference modeling. So the paper, it's two papers, it's called Realizing Synthetic Active Inference Agents. And they wrote, with a full message passing account of synthetic active inference agents, it becomes possible to derive and reuse message updates across models and move closer to industrial applications of synthetic active inference framework. So how does knowing the message passing structure help reuse a model across different settings or facilitate the legibility of the model?

Josef Pérez, Ph.D.: Right. So first of all, I'm not super familiar with the kind of active inference literature. So, you know, you'll have to help me there a bit for it to anchor maybe the discussions a bit more into that. But I think if I understand correctly, the kind of question you're getting at is basically how can kind of message passing help us generalize, you know, in kind of various kinds of settings or maybe, you know, from one graph to another, and things like that. And this is an kind of active area of research how exactly this generalization is happening. But something you could notice for is something that, for instance, was shown, like these models are quite good at, for instance, spotting patterns, or structures, depending on how exactly you implement them. But for instance, you know, let's say you have a triangle in your graph, or, you know, that'd be kind of the simplest structure, right? You

have a triangle, or some other kind of gadgets in your graph, like particular subgraphs, right, that might show up in different kinds of, you know, various graphs, the graphs themselves might look completely different from each other. But these kind of patterns might kind of be reemerging in multiple, like local patterns, might reemerge in multiple graphs. And that could help your way to kind of generalize, right? Like you could see, for instance, if you have clicks, they're super important in kind of when you do social network modeling and things like that, because they kind of show this kind of, you know, close group of friends, right, they all talk to each other. So they kind of form a click, like everyone's connected to each other, right. And then you might be able to use that in general, like another completely different social context, where, you know, these agents are again kind of communicating in a similar manner or connected in a similar manner, even if the kind of overall pattern is quite different. And it goes way beyond just kind of structural similarities, because there's also features in there. So there's combinations of kind of structural patterns and features that, you know, give you even more complicated patterns, right? Like you might have a triangle, but then also two of the features in this triangle are, you know, look in a certain way and one looks in another way. So that gives you even more kind of refinement and, you know, even kind of richer pattern detection abilities. So you have essentially this, you know, ability to kind of spot patterns at multiple scales as well. So you could see this happening at multiple scales, you could have patterns of patterns, right? You could have a community, entire communities connected in various patterns and so on. And again, it's kind of also a research question, how do you capture these hierarchical patterns and so on?

You know, in general, you have to do more message passing if you want to capture things that are further away from each other, because otherwise they can't talk to each other, right? So yeah, I don't know if that that actually answered your question or if I was kind of going in the right direction there.

Oh, that's great. It brings up a lot of different cool ideas, like this patterns all the way down, but totally agree. I think we can now perhaps explore some more specific connections to active inference because hopefully the listenership or viewership of this, it's kind of like a two-way street. Like some people may be coming from more of your background and then learning about active inference and generative models as a specific system of interest the first time, but also certainly for a lot of people in the active inference space, like these methods coming from category theory have only recently come up to, I guess, more prominence in Bayesian modeling, at least where we are. So it's kind of, it's a cool connection to make. I think one of the biggest touch points off the bat was like you mentioned, multiplying a matrix by a vector and interpreting that as an edge. So just in the, in the inference part of the generative model of about sensory observations, we always talk about the thermometer observation and an underlying hidden state temperature. So that exactly describes that case.

And that's why we can represent the active inference generative models, the perceptual parts and the action parts in terms of matrix multiplication. It's why the MATLAB code for generative models does look mostly like matrix multiplication and it can all be done explicitly that way. So how does that feel? Like are there models that don't have this feature or what do we gain by having like all of our edges

defined as a pre-sheave action with a matrix and a vector in this setting of agent generative models with perception and action? Like what? Yeah. Yeah. Any thoughts on that?

Yeah. I think essentially kind of the, the graph structure is kind of telling you these things interact in some way. Right. So, um, there, there's some communication between these vertices, if you, you know, if we're kind of in a, in a graph setting, right. Um, and then kind of what the sheath is giving you or any message passing model essentially is, is expressing a way that in which way that connection should manifest in the model or in what way that connection should be used to process information. Um, so in this case, I was mentioning, okay, we, you have like linear maps because you could go on the type, if your type of data are vector spaces, then you, this transformation will be some sort of linear maps. Um, but it doesn't necessarily have to be. So for instance, it could go to any nonlinear transformation, right? Like if, and this is what's happening in general in practice. Um, you know, if you, if you have a neighbor, the message coming from that neighbor could be modulated by any sort of transformation you want. So it could be linear, it could be nonlinear. It could be something, I don't know, you, you, you can specify it basically. Uh, but essentially you could think of this as you have a structure level telling you who should communicate to whom. Um, and then you kind of have some semantics that is kind of shift is adding on top, uh, saying how should these things communicate, right? Like the, the first thing is, uh, who should communicate or what should communicate. And then the semantics we add on top essentially describe, uh, how should that communication, uh, manifest, uh, essentially.

Very cool. I think that maps like exactly to how we talk about the sparsity of variables in the generative model. So here, the topology of the nodes in the graph that we want to do a message passing on are going to be describing like the agent and the environment or like the generative model that includes perception, cognition, and action. So a lot of people have proposed different sparsity architectures for integrated modeling of perception, cognition, action. So one example would just be like kind of around the clock, like action influences the environment, environment influences perception, back to cognition. You could add a self loop using our arc of blanket and different kinds of connectivities. And that defines the sparsity topologically, which is where you showed the stack and, and you were on the second and the third levels. Um, I think of the stack and then like what flows, it has to be described how it actually, what that edge does. So what, what is that? What is that?

That is also being provided. Yeah, yeah, exactly. And it could even go to the extreme where, you know, does that edge actually do anything? So for instance, if you, uh, have a matrix, that's just the zero matrix, for instance, associated to that edge, you were just going to multiply by zero, right? And that gives you zero and it's kind of essentially pruning that edge, right? Like I, I can't get rid of it. I don't want that communication to happen. So there's kind of, there's also kind of these possibility where these kinds of semantics, they override the structural level where you say, okay, I don't need to communicate, you know, with this other agent, person or whatever, it depends on what these vertices actually mean and in what context,

in what context you are. And then there's also the case where you could do some sort of selective pruning where, um, this matrix depending on, um, so in, in kind of linear algebra, the matrix has a kernel. So it's, uh, all the stuff that, that matrix sends to zero. So what, what vectors are sent to zero, right? But not everything will be sent to zero unless you're the zero matrix.

Um, so depending on the features of the neighbors, you could also just send some of the neighbors to zero, right? And that kind of removes those neighbors from the equation. They just kind of get, you know, you, you're, yeah, they're not, they're not factored in anymore. So you kind of have this, you know, it's a way to get the sparsity, I guess that, that you were also talking about, uh, where, you know, certain only maybe a small subset of the inputs or kind of, um, yeah, only, uh, uh, subset of the features are actually kind of doing some meaningful stuff, uh, among the neighbors and everything else will be kind of zeroed up.

Yeah. That makes me think of the lasso regression, which tries to set most variables of having an impact of zero so that a few hopefully important variables really pop out in the analysis, but also there's, there's newer techniques, I guess of attention modeling and re-weighting that isn't just like, okay, set five of them to one and then the rest, and then the rest of them to zero, like more nuanced. Um, so I think that sparsity with the expressivity is basically the, the best of both worlds, because you do want to have a situation where there is an edge, but the attention being paid to it is zero. So functionally that doesn't have an update on the belief state, even though the, in principle, the edge exists. And that's why we can model situations where like the agent believes they have impact in the world, but actually just because the edge in principle exists doesn't mean that it has any given impact. And so that allows like the articulation of these models where they factorize and keeps interpretable motifs in terms of just little clusters of, uh, motifs here in our case, describing the action perception and cognition types of systems of interest. But people, I believe already implicitly do this, um, like they will often add an adjective and refer to X kind of active inference. So like deep active inference with a temporal horizon, sophisticated active inference with this kind of nesting. And those are pointing to a given feature, but of course those features as we're hoping should be composable. And so this seems to be bringing tools that are even more general than just action perception modeling because they're at a lower level of abstraction than like any specific system of interest, but where this work and kind of timeless thinking around cybernetic systems come together through the active inference generative model as a Bayes graph, it gets very exciting.

Yeah. Yeah. And maybe it's also something worth emphasizing here is that even if this kind of semantic level can, can get rid of some edges, right? By doing this kind of pruning, something it cannot get rid of is the computation. So something that kind of that structural graph level, uh, forces you to do, it kind of tells you what should you spend compute time on, right? Cause like if, even if you're going to decide to prune an edge, you still need to decide that, which, you know, takes compute time. So you need, still need to look at all your neighbors if you're a node, right? And, and decide what to prune or maybe you don't prune anything or whatever, but you have to look at every edge. And one way to look at this is,

uh, the kind of the graph structure defines your computational graph or kind of a computational, yeah, a series of computational steps you have to execute. And then the kind of the shift structure or the message passing model actually specifies what those steps are and, you know, in what particular way they, they look exactly. Um, and so, yeah, that, that's one point. And yeah, you also mentioned attention and actually, yeah, I'm glad you did, cause this is actually quite related and in, in certain ways more general than attention. And actually maybe going back to this slide, it might be a nice way to, to see this. So here basically what happens in attention, um, instead of learning these matrices that we learned here, uh, in attention, you learn attention coefficients here. So you just learn a scalar. That's the attention coefficient. How much attention should I pay to essentially, well, you know, these overall edge, let's say, um, which will be just a scalar. What we do is kind of a bit more complicated because you just learn how do I transform these neighbors. So it's kind of a whole matrix rather than a single scalar. Um, uh, but there's also some subtle differences, but in, in, uh, in a follow-up work with it, we also combine this with, with attention and, and went a bit more general. And, um, that, that also worked, uh, quite well, but the kind of underlying idea is, is, is very similar. You want to modulate the way it transform information based on the information itself, right? So you have this kind of one level of, um, recursivity if you want to kind of, yeah, that, that we're also alluding to in that, you know, it happens in active inference where, okay, you're, you're, so if I'm node v , right? My neighbor u , it has some features and based on these features, which are x_u , I'm going to find out the matrix that will be used to process x_u , right? So it's kind of very recursive and it's what happens with attention, right? Based on the features of node u , I'm going to compute an attention coefficient that I'm going to apply to this feature of u , right? I'm going to decide based on this feature, how much attention should I pay to it? Um, and here we decide how should I process it more generally, uh, in a linear way. Um, so you have this kind of loopiness structure embedded in there. Awesome. I'll, I'll bring up a few more points because I think there's, there's so many great pieces. So Toby Sinclair Smyth, who we recently discussed his dissertation in Livestream 54, introduced a term or at least a phrasing the compositional cognitive cartography. And so thinking about the compositionality of cognitive systems. And I think what you're describing here with this notion that the mappings are more general than the kind of attention mechanisms known famously today, that those represent like a lower dimensional special case of one kind of architecture. makes me think about how the Bayesian graph is kind of semantic in principle and can have all of these nice categorical formalisms around them, but then, and you can even build the connector to empirical data with a pre-sheaf and the sheaf, which may be news to even many empirical researchers doing data analysis certainly was for me, but the message passing provides a rigorous translation from whatever semantic model is proposed topologically to an implementation procedure that can be planned for and executed in linear time, or at least with definable characteristics. So message passing plays a really important part in going from like the abstract, what is possible to the implementations of any of these actual models. And it does it in a really general way where is it accurate to say that we hope that implementation with message passing compatible generative models will kind of roll out better

because we won't have some of the engineering challenges

changes that less reusable abstractions might carry.

It's hard to say. I think there's also certainly some limitations to this paradigm

as well. So just kind of doing this kind of message passing thing, as you were mentioning, one thing is that it kind of scales up quite easily, like linearly with the size of the graph, but that also come at the cost. So there's certain results showing this has limits in expressivity.

So if you actually want to go beyond this, for instance, you have to, instead of just looking at pairs of nodes, you have to look at tuples and these kind of high order groupings of nodes in order to kind of get higher expressivity. There's all sorts of techniques to do that.

But yeah, and there's always this kind of tension between being more expressive and being efficient that will always be there in any sort of algorithm or method.

So it's kind of hard to say. I mean, it's definitely, we can definitely say this is kind of not the ultimate solution, let's say, if you want to do things, message passing in itself, you know, but, you know, maybe doing some sort of computations on graphs, you know, could be.

Maybe also something that's kind of missing a bit in kind of the graph ML setting is, you know, the context where, you know, your graph is, you assume your graph is known.

And, you know, you need to have some graph structure, at least a sensible way to construct it, right.

But for many kind of more, I don't know, like, more, how should I phrase that?

I guess for less clearly defined things, like, okay, if I'm an agent doing perception in the real world or something, right, like, if I'm trying to create a graph of the world, you know, like, what's an object?

What do I create? What do I create a node for, right?

If I want to have, like, one node per object, and there's some connections between objects and things like that, just, I know, it's like some wild example that comes to mind.

I don't know if you actually want to do that, but let's say you want, right, then there's also, like, all these kind of blurry things, like, what's an object and what's not an object?

Like, what's kind of somewhere in between, maybe, you know, is that a node?

Is that, so it's kind of like, what I'm trying to say that the graph structure is kind of very discreet, right?

Like, it's kind of the node is either there or it's not there, and edge is there or it's not there.

But then the world is kind of very fuzzy, right?

So if you use graphs as a model for your world, then there probably has to be some decision to be made somewhere about these kind of fuzzy concepts.

Do they actually translate in a concrete graph entity, like an object, an edge, or whatever, or not, based on, you know, some kind of inference procedure?

And I don't know if we did that or not, as kind of humans, as kind of as intelligent agents, but that's, you know, kind of some interesting thing to think about.

Maybe you could also, well, maybe one way to solve that is also kind of stuff like soft edges and things like that.

And in some way, you know, if you have attention coefficients, it's a bit like that, you know, like if an edge has a weight of 0.001 or something, it's almost like not being there, but, you know, it's still kind of there.

So it's, you know, it's a bit of a soft graph architecture.

But so I guess at the edge level, you can implement this softness, but I think it's a bit harder at kind of the node level, right?

Like how do you kind of model a node that's kind of there and not there?

Yeah, there's just some random thoughts.

That's very interesting about the fuzzy object identification and kind of similarities and differences between nodes and edges,

even though in some ways they have some similarities to or interoperabilities to.

One other point of contact was like an underlying hidden space that we understand topologically that projects a vector space at it from different places.

So that could be a vector of thermometer readings.

And we want to have a smooth path within the homeostatic range defined up to a boundary point.

Not saying that that's the structure of the world, but a structure of a very heuristic and simple model might be to aim for continuity

and have a defined hidden state space that has continuity underneath and is able to emit vectors.

So that kind of brings some of these classifier type discussions that you brought up

and like the kind of fundamental impossibility of geometric classification because you are going to end up with gray zones.

Whereas even if it takes a bitwise description, you can separate the network.

So that gives an actual completeness measure and that allows measures like, I mean, amount of computational resources

or in a more like statistically principled way, like the Bayesian information criterion.

So how many nodes should we have?

We should be on some trade-off front in some modeling space.

I don't know what to tell you.

It's a map, not the territory.

And that's because that's more justifiable.

And so even lifelike organisms might want to self-evidence staying emitting from a living state.

And so that provides a really simple graphical architecture to cybernetic systems.

And then Active Inference explores a lot of different, more specific motifs within that broader blanket persistence picture and the path of least action.

So that's what enables the physics in that space and why these methods, which as far as I understand, are often used in quantum mechanics, are being able to come together with Active Inference this way.

Yeah, something that comes to mind when you mentioned this, I think there's also like a recent avenue of research in this area where people,

and it's again kind of generated by the fact that you don't know the graph beforehand many times,

And I think kind of the old school approach was, well, you construct it based on some rules, right?

Like you're going to say, oh, this, I don't know, some things are similar.

I'm going to put an edge between them and you define similar in whatever way you like and so on.

And there was this kind of recent trend where what you try to do is kind of latent graph inference,

or some people call it manifold learning.

If you think of the graph as some sort of manifold, but this kind of very informally speaking.

And essentially what you would do is like you would map whatever you try to learn, the raw observations into some latent space.

And that's where you actually construct the graph.

You construct the graph in the latent space rather than kind of in the raw space.

So that might be kind of a way to deal with buzziness as well, right?

Like, because then I guess you might lose some of these kind of very concrete one-to-one mappings because you might learn some node in the latent space that maybe corresponds to three or four concepts kind of mixed together.

And, you know, there's all these kind of nice experiments with neurons in deep networks kind of visualized.

And, you know, they learn maybe a kind of a mixture of concepts.

Like, if you see what actually, you know, activates that neuron is actually, you know, maybe a few classes or different kinds of things.

It's not necessarily a single thing.

So it could be something very similar here where you have some very entangled representations that are kind of distilled in this latent graph.

And then, you know, at least in concept space, even if you're kind of in the latent space,

that's still like kind of a very, you know, clear combinatorial structure

with respect to kind of your raw observations,

that structure kind of can still kind of encode the fuzziness of the world to some degree

because you have this kind of mixture of concepts that got distilled in the same node

or things like that.

Or maybe some concepts could be represented by multiple nodes,

depending on, you know, in what way you see these concepts.

There might be all sorts of variations per concept or points of view and so on.

So I think, you know, kind of latent graph inference could be, you know,

quite an interesting way maybe to address some of these issues we were discussing.

Although I think it kind of died off a bit in the recent year,

at least as far as I've seen, there were a few, you know,

slightly fewer papers on the topic.

Well, certainly the agents proposed latent structure of the world,

the causal structure of the world is just mapped out a territory.

And so it enables maybe some of those core screenings.

Could you go to the slide where there was a mapping between a smooth sphere

and then a regular geometric shape?

Yeah.

Let's see.

In this one?

Yeah.

Just wanted to make one point and see if you had any comments.

At the heart of some of the relationships that you're describing and where you pulled back to in terms of generalization, helps us understand this relationship between the sphere and the geometry.

And one place that that and the implications for data processing and all of the computational science areas is if you are preserving or learning or analyzing geometry, but not topology, or the other way around, you might get these different like data set aberrations.

Like you might have the topology of the coffee cup, but it looks like something totally different.

And so what we would really want to do would be understand the relationship between geometry and topology.

Because if we could understand it in principle, like you have it on the left side, and then in practice with the data scheme on the right side, or insert your own left and right side there, then we'd be able to do data analysis in a way that respected slash preserved both the topology and the geometry.

So it's like two compatible perspectives that have their different like strengths and weaknesses and heuristics.

And so understanding that relationship between geometry and topology and the implicit spaces that geometry requires and so on, that has tremendous use.

And just in closing reminds me of Buckminster Fuller's Synergetics, which uses a close packing architecture and a tetrahedron centric model of coordinates to find more continuity between surface area and volume and between the smooth surfaces and the great circles on them and like the points of connectivity on shapes.

So I think it's an incredibly deep area and really has fundamental impact in active inference.

Helps us think about our models in this way, kind of like the inflated balloon and with the fuzziness and the architecture and the finiteness.

It really brings a lot to active inference.

And so I appreciate you sharing the work with us today
and continuing to work in this way.

Thanks a lot.

Any last thoughts?

Yeah, I think, yeah, what you mentioned,

I think it's been all over my thesis,
this kind of tension between topology and geometry.

And maybe what I want to emphasize is that,
you know, I'm not saying kind of the previous perspective
of looking maybe more geometrically at things was wrong in any way.

And on the contrary, actually,
there's lots of interesting places where these things intersect,
even in kind of this sheath paper I briefly went through.

Like if you actually read the paper,
there's a lot of beautiful intersections.

Actually, my main collaborator, Francesco,
he's a differential geometer.

So he actually had lots of kind of inputs from that side.

And indeed, I think, you know,
we should try to use all these kind of layers of structure,
you know, in the best way possible for all our methods.

Awesome.

All right.

Thank you.

Until next time.

Thanks for having me.

Bye.

Thank you.